GMX · SECURITY REVIEW

GMX
Just-in-Time Liquidity.

A security review of Just-in-Time Liquidity.

CLIENT	PROJECT	AUDITORS	AUDIT WINDOW	FINAL REPORT
GMX	Just-in-Time Liquidity	Guardian	August 27 – November 5, 2025	November 5, 2025

§ 00

Contents.

63 pages: overview, confidence ranking, scope, 50 findings, disclaimer, and about Guardian.

§ 01 **Overview** P. 03

§ 02 **Confidence Ranking** P. 04

§ 03 **Scope & Classifications** P. 05

§ 04 **Findings** P. 06

Main Review · 41 findings P. 09

Remediation Review 1 · 6 findings P. 53

Remediation Review 3 · 3 findings P. 59

§ 05 **Disclaimer** P. 62

§ 06 **About Guardian** P. 63

§ 01

Overview.

Guardian reviewed Just-in-Time Liquidity across 4 review rounds. The final report includes 2 High, 13 Medium, 34 Low, 1 Informational findings.

AUDIT FIRM
Guardian

CLIENT
GMX

PROJECT
Just-in-Time Liquidity

PREPARED BY
Guardian

AUDIT WINDOW
August 27 – November 5, 2025

FINAL REPORT
November 5, 2025

REPOSITORY
https://github.com/gmx-io/gmx-synthetics

METHODOLOGY
Smart contract · Manual · Main Review · Remediation Review 1 · Remediation Review 2 · Remediation Review 3

MAIN REVIEW COMMIT
1be5f5166fff

REMEDIATION REVIEW 1 COMMIT
13ce328959bf

REMEDIATION REVIEW 2 COMMIT
b2d61e4a4825

REMEDIATION REVIEW 3 COMMIT
708171048fd2

1.1 Confidence Ranking

4 / 5

High Confidence

Guardian assigns a Confidence Ranking of 4 out of 5, combining finding severity, remediation outcome, and the reviewer's assessment of latent risk. The full scale is on the next page.

1	2	3	4	5
VERY LOW CONFIDENCE	LOW CONFIDENCE	MODERATE CONFIDENCE	**HIGH CONFIDENCE**	VERY HIGH CONFIDENCE

1.2 Findings at a Glance

SEVERITY	TOTAL	PENDING	ACK	PARTIAL	RESOLVED
Critical	**0**	0	0	0	0
High	**2**	0	0	0	2
Medium	**13**	0	5	1	7
Low	**34**	0	19	1	14
Informational	**1**	0	1	0	0
All severities	**50**	**0**	**25**	**2**	**23**

§ 02

Confidence Ranking.

Guardian assigns a project-level confidence score using the scale below. The ranking combines finding severity, remediation outcome, and the reviewer's assessment of latent risk.

CONFIDENCE RANKING	DEFINITION AND RECOMMENDATION	RISK PROFILE
5 Very High Confidence	Codebase is mature, clean, and secure. No High or Critical vulnerabilities were found. Follows modern best practices with high test coverage and thoughtful design. Recommendation: code is highly secure at time of audit.	0 High/Critical findings and few Low/Medium severity findings.
4 High Confidence ASSIGNED	Code is clean, well-structured, and adheres to best practices. Design patterns are sound and test coverage is strong. Recommendation: suitable for deployment after remediations; consider periodic review with changes.	0-1 High/Critical findings per engagement week and little to no Medium severity findings.
3 Moderate Confidence	Code is functional but shows gaps in defensive programming or test coverage. Recommendation: remediate the findings and re-review the changes before deployment.	Some High/Critical findings, with various Medium severity findings.
2 Low Confidence	Recurring issues suggest systemic gaps rather than isolated mistakes. Recommendation: remediate, then commission a follow-up review at a frozen commit.	Over 6 High and Critical findings across the engagement.
1 Very Low Confidence	The volume and severity of findings indicate the codebase is not ready for production. Recommendation: substantial rework, then a full re-review.	Over 12 High and Critical findings across the engagement.

§ 03

Scope & Classifications.

Guardian reviewed Just-in-Time Liquidity in `https://github.com/gmx-io/gmx-synthetics`.

3.1 Severity Matrix

	IMP · HIGH	IMP · MED	IMP · LOW
LIK · HIGH	Critical	High	Medium
LIK · MED	High	Medium	Low
LIK · LOW	Medium	Low	Low

IMPACT · HIGH	Significant loss of assets, harm to a group of users, or core protocol functionality is disrupted.
IMPACT · MEDIUM	Small amount of funds can be lost, or ancillary functionality is affected. Users may experience reduced or delayed receipt.
IMPACT · LOW	Unexpected behavior in some functionality that is notable but does not meet the criteria for higher severity.
LIKELIHOOD · HIGH	Possible with reasonable assumptions that mimic on-chain conditions; cost is low compared to amount gained.
LIKELIHOOD · MEDIUM	Possible only in uncommon cases or requires a large amount of capital relative to disruption caused.
LIKELIHOOD · LOW	Unlikely to ever occur in production.

§ 04

Findings.

Grouped by round. Visible IDs reset by severity within each round, so a finding is named the same way in the report as it was in the channel it was written in. Status is where each finding ended up; its own page says how.

ID	TITLE	CATEGORY	SEVERITY	STATUS	PAGE
MAIN REVIEW					
H-01	Keeper Fees Are Trapped	Logical Error	HIGH	RESOLVED	p. 9
H-02	Limit Orders Require Out Of Date Prices	Logical Error	HIGH	RESOLVED	p. 10
M-01	Missing Feature Validation	Validation	MEDIUM	RESOLVED	p. 11
M-02	Orders Execution Fee Does Not Cover Shifts	Validation	MEDIUM	ACKNOWLEDGED	p. 12
M-03	Inaccurate Gas Validations	Validation	MEDIUM	RESOLVED	p. 13
M-04	Users Only Pay Gas For Shifts On Failure	Logical Error	MEDIUM	RESOLVED	p. 15
M-05	Misleading Simulation Results	Unexpected Behavior	MEDIUM	RESOLVED	p. 16
M-06	poolToPnlRatio Abuse With JIT	Gaming	MEDIUM	ACKNOWLEDGED	p. 17
M-07	GLV Oracle Lag Causes Accounting Mismatch	Gaming	MEDIUM	ACKNOWLEDGED	p. 18
M-08	GLV Arbitraged With JIT	Gaming	MEDIUM	PARTIALLY RESOLVED	p. 20
M-09	Missing Keeper Error	Validation	MEDIUM	RESOLVED	p. 22
M-10	GLV Used To Exit Illiquid Markets	Gaming	MEDIUM	ACKNOWLEDGED	p. 23
M-11	Target Shifts Can Not Be Simulated	Logical Error	MEDIUM	RESOLVED	p. 24
L-01	Misleading Simulations Because Of Shared Key	Warning	LOW	ACKNOWLEDGED	p. 25
L-02	ADLs Executed When Orders Disabled	Warning	LOW	ACKNOWLEDGED	p. 26
L-03	Unused getGlvTokenPrice Function	Superfluous Code	LOW	RESOLVED	p. 27
L-04	Lacking Reentrancy Checks	Best Practices	LOW	ACKNOWLEDGED	p. 28
L-05	Lacking From And To Market Check	Validation	LOW	RESOLVED	p. 29
L-06	Outdated Naming	Documentation	LOW	RESOLVED	p. 30
L-07	Unexpected Behavior When GLV First Created	Warning	LOW	ACKNOWLEDGED	p. 31
L-08	Max Pnl Validation Missed	Validation	LOW	RESOLVED	p. 32
L-09	Increased Risk Of Arbitrage	Gaming	LOW	ACKNOWLEDGED	p. 33
L-10	GlvShift Interval DoS	DoS	LOW	ACKNOWLEDGED	p. 34
L-11	Unclear getPrimaryPrice Behavior	Documentation	LOW	RESOLVED	p. 35
L-12	GLV Value Updates Are Incorrect	Logical Error	LOW	ACKNOWLEDGED	p. 36

Findings continued.

ID	TITLE	CATEGORY	SEVERITY	STATUS	PAGE
L-13	Unexpected Key Replay	Unexpected Behavior	LOW	RESOLVED	p. 37
L-14	Shifts Occur No Matter The Order Result	Unexpected Behavior	LOW	RESOLVED	p. 38
L-15	Multiple Shifts Cannot Occur	Logical Error	LOW	RESOLVED	p. 39
L-16	Lacking doExecuteGlvShift Reentrancy Guard	Reentrancy	LOW	RESOLVED	p. 40
L-17	Order Execution Feature For Liquidation Contract	Unexpected Behavior	LOW	ACKNOWLEDGED	p. 41
L-18	Unnecessary GLV Price Timestamp Requirements	Unexpected Behavior	LOW	ACKNOWLEDGED	p. 42
L-19	Unexpected Order Cancellation	Unexpected Behavior	LOW	ACKNOWLEDGED	p. 43
L-20	JIT DoS'd With Atomic Actions	DoS	LOW	ACKNOWLEDGED	p. 44
L-21	JIT Short Term Risk Free Trades	Gaming	LOW	RESOLVED	p. 45
L-22	JIT May Cause Some Orders To Fail	Unexpected Behavior	LOW	ACKNOWLEDGED	p. 46
L-23	JIT May Prevent Withdrawals	Warning	LOW	ACKNOWLEDGED	p. 47
L-24	GLV May Be Left Improperly Allocated	Warning	LOW	ACKNOWLEDGED	p. 48
L-25	Unnecessary ReentrancyGuard	Superfluous Code	LOW	RESOLVED	p. 49
L-26	Multiple Shifts From A Market	Validation	LOW	ACKNOWLEDGED	p. 50
L-27	Unnecessary Library Usage	Gas Optimization	LOW	RESOLVED	p. 51
L-28	Simulation Misses Reentrancy Guard Close	Unexpected Behavior	LOW	RESOLVED	p. 52

REMEDIATION REVIEW 1

M-01	GLV Shift Misses Feature Validation	Validation	MEDIUM	RESOLVED	p. 53
L-01	Remaining ReentrancyGuard Import	Best Practices	LOW	RESOLVED	p. 54
L-02	GLV Oracle Inaccuracy	Logical Error	LOW	PARTIALLY RESOLVED	p. 55
L-03	onlySelfOrController Unnecessary	Superfluous Code	LOW	ACKNOWLEDGED	p. 56
L-04	Integration Risk During Upgrade	Warning	LOW	ACKNOWLEDGED	p. 57
L-05	Increased Hedge Risk For GLV	Warning	LOW	ACKNOWLEDGED	p. 58

REMEDIATION REVIEW 3

M-01	Manual Cancellation Sends Refund To Unexpected Address	Unexpected Behavior	MEDIUM	ACKNOWLEDGED	p. 59
L-01	Swap Oracle Requirement Understated	Warning	LOW	ACKNOWLEDGED	p. 60

Findings continued.

ID	TITLE	CATEGORY	SEVERITY	STATUS	PAGE
I-01	Single Market GLVs Do Not Collect Enough Fees	Warning	INFORMATIONAL	ACKNOWLEDGED	p. 61

HIGH **H-01** RESOLVED

Keeper Fees Are Trapped

CATEGORY	Logical Error
LOCATION	`JitOrderHandler.sol`
REVIEW	Main Review

DESCRIPTION

In the `executeOrderFromController` function the keeper is assigned as the `msg.sender`, however the caller of this function will be the `JitOrderHandler`.
As a result the keeper fees will be sent to the `JitOrderHandler` contract and trapped.
The same will occur for the execution fee upon cancellation or freezing since the `_handleOrderError` assigns the `msg.sender` as the keeper always in the invocations to `OrderUtils.cancelOrder` and `OrderUtils.freezeOrder`.

RECOMMENDATION

Add a keeper parameter to the `executeOrderFromController` function.

RESOLUTION

Status: Resolved.

HIGH **H-02** RESOLVED

Limit Orders Require Out Of Date Prices

CATEGORY	Logical Error
LOCATION	`JitOrderHandler.sol`
REVIEW	Main Review

DESCRIPTION

When executing a `LimitIncrease` or `StopIncrease` order through the `JitOrderHandler` contract, the `updatedAtTime` of the `GlvShift` action is set to the `orderUpdatedAtTime`. However in the case of `LimitIncrease` or `StopIncrease` orders, the `orderUpdatedAtTime` will often be far in the past.
The logic for deposits and withdraws which will occur as a part of the GLV shift requires that the prices set in the oracle come from around the time of the action's `updatedAtTime`. As a result, if a keeper were to execute this action with prices from the order's creation the prices would be invalid for the current market value of GM markets.
In practice, this simply reverts due to failing the oracle's maximum staleness checks, so `LimitIncrease` and `StopIncrease` orders cannot be executed with JIT.

RECOMMENDATION

Consider allowing the order keeper to provide two sets of prices, one for the Shift action and one for the order action, ensuring that prices are cleared after the shifts are executed and again after the order is executed.
This way the difference in price requirements can be met no matter what. The shift action should use the most up to date prices possible, using the `block.timestamp` as the `updatedAtTime` similar to atomic withdrawals. While the order action may have specific requirements on when the prices are from depending on when the trigger price was hit.
Also notice that the `simulateExecuteJitOrder` function will have to be updated to handle two separate price submissions.

RESOLUTION

Status: Resolved.

MEDIUM **M-01** RESOLVED

Missing Feature Validation

CATEGORY	Validation
LOCATION	ExecuteGlvDepositUtils.sol
REVIEW	Main Review

DESCRIPTION

In the `_processMarketDeposit` function during a GLV deposit, there is no validation to ensure that the GM market has deposits enabled before executing the deposit. This is in contrast to the way that Shift actions execute deposits using the `executeDepositFromController` function which validates the deposit feature. In the `_processMarketWithdrawal` function during a GLV withdrawal, there is no validation to ensure that the GM market has withdrawals enabled before executing the withdrawal. This is in contrast to the way that Shift actions execute deposits using the `executeWithdrawalFromController` function which validates the withdrawal feature.

RECOMMENDATION

Consider validating that the withdrawal feature is enabled during these functions, or using the `executeDepositFromController` and `executeWithdrawalFromController` functions on the respective handlers.

RESOLUTION

Status: Resolved.

MEDIUM M-02

ACKNOWLEDGED

Orders Execution Fee Does Not Cover Shifts

CATEGORY	Validation
LOCATION	`JitOrderHandler.sol`
REVIEW	Main Review

DESCRIPTION

Jit orders now require the execution of several shifts, the gas needed for which will be tracked by the `startingGas` recorded in the `executeJitOrder` function.
However the order execution fee is not increased for these orders, as a result the keeper will often subsidize the gas consumption of these shifts.

RECOMMENDATION

Be aware of this behavior and consider if it is acceptable for the keeper to subsidize the gas costs for these shifts.

RESOLUTION

Status: Acknowledged.

MEDIUM **M-03** RESOLVED

Inaccurate Gas Validations

CATEGORY	Validation
LOCATION	`JitOrderHandler.sol`
REVIEW	Main Review

DESCRIPTION

In the `executeJitOrder` function of the `JitOrderHandler` contract the `validateExecutionGas` and `getExecutionGas` validations are performed on the `gasleft()` value.
However this gas value does not match the gas that is available in the `executeOrderFromController` mostly due to the 63/64 rule for the external call to the `OrderHandler`.
Therefore the gas available to the `_executeOrder` function is not accurately validated.

RECOMMENDATION

Consider moving the gas validations into the `executeOrderFromController` function similarly to the `executeOrder` function such as the following:

```javascript
function executeOrderFromController(
bytes32 key,
Order.Props memory order,
uint256 startingGas,
bool isSimulation
) external onlyController {
uint256 executionGas = gasleft();

uint256 estimatedGasLimit = GasUtils.estimateExecuteOrderGasLimit(dataStore, order);
GasUtils.validateExecutionGas(dataStore, executionGas, estimatedGasLimit);

executionGas = GasUtils.getExecutionGas(dataStore, executionGas);

try this._executeOrder{ gas: executionGas }(
key,
order,
msg.sender,
isSimulation
) {
} catch (bytes memory reasonBytes) {
```

M-03 Inaccurate Gas Validations CONTINUED

```
_handleOrderError(key, startingGas, reasonBytes);
}
}
```

RESOLUTION

Status: Resolved.

MEDIUM **M-04** RESOLVED

Users Only Pay Gas For Shifts On Failure

CATEGORY	Logical Error
LOCATION	`JitOrderHandler.sol`
REVIEW	Main Review

DESCRIPTION

In the `executeJitOrder` function the startingGas is declared before the shifts are processed in order to track the amount of gas used by the shift actions for the JIT order.
However the startingGas value passed to the `executeOrderFromController` function is only used when the order execution fails. Upon the successful completion of an order the startingGas that is re-declared at the beginning of the `_executeOrder` function is used, which does not track the gas usage of the previous shift actions.
As a result users with orders that successfully complete with JIT will not track the gas expenditure made for the shifts and may end up refunding users a portion of their execution fee which ought to have paid the keeper for the shift gas expenditure.

RECOMMENDATION

Consider updating the `_executeOrder` function to accept a `startingGas` value as a parameter rather than declaring it at the beginning of the function.

RESOLUTION

Status: Resolved.

MEDIUM M-05 RESOLVED

Misleading Simulation Results

CATEGORY	Unexpected Behavior
LOCATION	`JitOrderHandler.sol`
REVIEW	Main Review

DESCRIPTION

In the `simulateExecuteJitOrder` function the shared `orderHandler.executeOrderFromController` function is used to execute the simulation. However this function includes a try/catch which will prevent any error encountered during the order's execution from being raised.
This is in contrast to the existing `simulateExecuteOrder` function which does not include a try/catch.

RECOMMENDATION

Create a separate `simulateExecuteOrderFromController` function to be used for the `simulateExecuteJitOrder` function which does not include a try/catch.

RESOLUTION

Status: Resolved.

MEDIUM **M-06** ACKNOWLEDGED

poolToPnlRatio Abuse With JIT

CATEGORY	Gaming
LOCATION	Global
REVIEW	Main Review

DESCRIPTION

Traders now have the ability to potentially influence where GLV shifts are made by creating large orders for distinct markets.
As a result, traders could potentially extract value by pushing GLV liquidity into markets that have a `pnlToPoolRatio` above the withdrawal or trader max pnl levels.
If a pool is above the withdrawal max `pnlToPoolRatio`, any user who holds GM tokens in that market is incentivized to create a large order and trigger JIT to force liquidity to shift into that market and allow them to withdraw their GM tokens at the expense of the GLV holders, who are now holding a nearly illiquid GM token.
If a pool is above the trader's max pnl `pnlToPoolRatio`, any user who has a position which is being profit capped is incentivized to open a large trade in that market to draw more liquidity in to decrease the `pnlToPoolRatio` and unlock more of their profits at the expense of the GLV holders.

RECOMMENDATION

Consider adding off-chain validations to avoid triggering Jit order executions for markets which have a `pnlToPoolRatio` above or even close to the withdrawal `pnlToPoolRatio`.

RESOLUTION

Status: Acknowledged.

MEDIUM M-07 ACKNOWLEDGED

GLV Oracle Lag Causes Accounting Mismatch

CATEGORY	Gaming
LOCATION	`GlvUtils.sol`
REVIEW	Main Review

DESCRIPTION

When using the oracle price for the `getGlvValue` result, the GLV price is lagging from a few seconds ago. However the price of the underlying market tokens which are subsequently used in the deposit or withdrawal actions, e.g. to figure out how many market tokens to withdraw, is from the current block.
This time differential between when the GLV price is measured from and when the underlying GM price is measured from results in mispricing of GLV which occurs over time.
The most obvious case of this is when a GM market experiences a stepwise jump in price. Though this is not necessary for this affect to take place over a period of time as small deviations build up and affect the GLV price.
For the sake of example we will examine the affect on GLV price when an underlying GM market experiences a stepwise decrease in price resulting from the insolvent liquidation of a position. The stepwise decrease in price occurs from the system realizing in that single liquidation transaction that the position could not pay for it's total negative PnL.

Consider the following scenario:
- GM A is priced at \$2 in block 100
- GLV holds just 5 GM A tokens
- GLV supply is 5
- GLV is worth \$10 in block 100
- User A withdraws 2 GLV to GM A and their GLV withdrawal is processed in block 105 using the GLV price from block 100 via the oracle
- GLV is valued at \$2 per GLV during the withdrawal, giving User A \$4 of withdrawal value
- GM A is valued at \$1 during the withdrawal, because it is using the valuation of the market token from block 105
- User A withdraws 4 GM A tokens from the GLV
- Now GLV price is 1 GM A token per 3 GLV = \$0.33

M-07 GLV Oracle Lag Causes Accounting Mismatch CONTINUED

- If User A had not done this withdrawal straddling the stepwise decrease in GM A price, the GLV price would rightfully be \$1.

Notice that a distinct stepwise decrease does not have to happen, but if minor deviations between these two times contribute to shifts in price over a long period of time GLV holders will have their earnings markedly affected.

RECOMMENDATION

Ideally, the price of the GM token being deposited and withdrawn would be valued from the same point in time as the oracle price for GLV.
Otherwise an imperfect solution would be to simply ensure that the block lag is consistently as small as possible for the GLV oracle, and hope that no notable stepwise jumps are straddled by a GLV action.

RESOLUTION

Status: Acknowledged.

MEDIUM **M-08** PARTIALLY RESOLVED

GLV Arbitraged With JIT

CATEGORY	Gaming
LOCATION	`JitOrderHandler.sol`
REVIEW	Main Review

DESCRIPTION

The JIT execution strategy can be triggered by users when they submit an order that exceeds the available liquidity in the pool. Given that users can trigger the exact market that JIT will move funds towards, it's possible for a user to frontrun the shift of a JIT action in order to benefit off of the liquidity amounts being deposited through price impact.
For example:
- GM A and B are in the same GLV
- The GLV holds 50 of the 100 GM A supply
- GM A has \$10,000 WETH and \$20,000 USDC
- GM B has \$15,000 WETH and \$15,000 USDC
- User A deposits \$5,000 of WETH and \$5,000 of USDC into GM B
- GM B now has \$20,000 WETH and \$20,000 USDC
- User A triggers a JIT action to shift from GM A to GM B by opening a large order for GM B
- GLV Shifts 50% of GM A into GM B
- GM B now has \$25,000 WETH and \$30,000 USDC, GLV experiences negative impact during this shift because GM B is now imbalanced
- User A withdraws \$2,500 WETH and \$7,500 USDC and receives positive impact because they have now balanced GM B back to \$22,500 WETH and \$22,500 USDC

There are maximum loss checks that take place upon GLV Shifts, however this only applies for a single GLV shift action. If an actor repeats this process many times over a period of time they can arbitrage much more than the maximum loss percentage over several shifts.
To make this strategy easily executable, users could continue to submit orders that trigger JIT but automatically fail their execution and are cancelled due to minimum output amount configurations on the increase order swap trivially failing to prevent the user from taking unwanted exposure for large orders.

RECOMMENDATION

Consider adding logic in the off-chain JIT keeper that prevents JIT actions from occurring when GLV Shifts have accumulated a certain threshold of losses due to price impact and

M-08 GLV Arbitraged With JIT

CONTINUED

other fees over a period of time.
Furthermore, consider simulating the result of the JIT action and ensuring that the order is successfully executed and not cancelled or frozen by the JIT execution.

RESOLUTION

Status: Partially resolved.

MEDIUM **M-09** RESOLVED

Missing Keeper Error

CATEGORY	Validation
LOCATION	`BaseHandler.sol`
REVIEW	Main Review

DESCRIPTION

In the `OrderUtils.cancelOrder` function the `InsufficientGasForAutoCancellation` error has been introduced which is raised instead of the `InsufficientGasForCancellation` error for non-autoCancel orders.
However the `InsufficientGasForCancellation` error has special handling where it causes a revert in the `OrderHandler._handleOrderError` function to prevent keepers from censoring orders. The `InsufficientGasForAutoCancellation` is not included in the `validateNonKeeperError` validation and therefore allows keepers to censor orders by providing insufficient gas for the auto cancellation order.

RECOMMENDATION

Include the `InsufficientGasForAutoCancellation` error in the `validateNonKeeperError` validation.

RESOLUTION

Status: Resolved.

MEDIUM **M-10** ACKNOWLEDGED

GLV Used To Exit Illiquid Markets

CATEGORY	Gaming
LOCATION	Global
REVIEW	Main Review

DESCRIPTION

With the new JIT feature there is a pathway for holders of GM tokens that are illiquid due to poolToPnl ratio to exit their GM holdings at the expense of GLV.
Consider the following scenario:
- User A holds 100 GM A
- GM A is at a pnlToPoolRatio of 30%
- The pnlToPoolRatioForWithdrawals is 20%
- GM A is currently locked out of withdrawals
- User A forces GLV to shift into GM A by creating a large trade in GM A
- The pnlToPoolRatio of GM A drops to 15% because User A's large position has no profit
- User A closes their large position
- User A withdraws their GM A tokens since now the pnlToPoolRatio allows them to do so
- As a result of User A's withdrawal, the pnlToPoolRatio goes back to 20%
- GLV is stuck with the now again illiquid GM A tokens

RECOMMENDATION

Be aware of this manipulation to exit illiquid GM markets. Consider adding validation to prevent such gaming based on the `pnlToPoolRatio` of markets that are being shifted into. No market that is the receiver of a `GlvShift` should be above the maximum `withdrawalPnlToPoolRatio`.

RESOLUTION

Status: Acknowledged.

MEDIUM **M-11** RESOLVED

Target Shifts Can Not Be Simulated

CATEGORY	Logical Error
LOCATION	`ShiftHandler.sol`
REVIEW	Main Review

DESCRIPTION

In the `ShiftHandler` contract the `simulateExecuteShift` function still has the `onlyController` modifier which means that users cannot simulate the execution of a particular shift.

RECOMMENDATION

Remove the `onlyController` modifier from the `simulateExecuteShift` function in the `ShiftHandler` contract.

RESOLUTION

Status: Resolved.

LOW **L-01** ACKNOWLEDGED

Misleading Simulations Because Of Shared Key

CATEGORY	Warning
LOCATION	ExchangeRouter.sol
REVIEW	Main Review

DESCRIPTION

The `simulateExecuteLatestJitOrder` function assumes that the current key belongs to an order, however since the key is shared across multiple actions such as orders, deposits, and withdrawals, the key may not belong to an order object.
This can lead to perturbed simulations when a deposit or withdrawal has been created after an order and the deposit or withdrawal is fetched from the dataStore in place of the latest order.

RECOMMENDATION

Be aware of this behavior and consider if it is desired in the simulation flow.

RESOLUTION

Status: Acknowledged.

LOW L-02

ACKNOWLEDGED

ADLs Executed When Orders Disabled

CATEGORY	Warning
LOCATION	`AdlHandler.sol`
REVIEW	Main Review

DESCRIPTION

The `AdlHandler` only validates if the `executeAdlFeatureDisabledKey` feature is disabled before executing an order with the `ExecuteOrderUtils.executeOrder` function. This ignores the `executeOrderFeatureDisabledKey` feature and is in contrast to the liquidation handler which validates the `executeOrderFeatureDisabledKey` feature.

RECOMMENDATION

Consider if this is the expected behavior and consider also validating the `executeOrderFeatureDisabledKey` feature in the `AdlHandler`.

RESOLUTION

Status: Acknowledged.

LOW L-03

RESOLVED

Unused getGlvTokenPrice Function

CATEGORY	Superfluous Code
LOCATION	`GlvUtils.sol`
REVIEW	Main Review

DESCRIPTION

The `getGlvTokenPrice` function which takes only 5 parameters defined here: https://github.com/gmx-io/gmx-synthetics/blob/1be5f5166fff8cd6016a8ce9d7b57ab93ded3870/contracts/glv/GlvUtils.sol#L145 is not used throughout the codebase.

RECOMMENDATION

Consider if this version of the function should be used instead of the following one which is used in the `GlvReader` contract.

RESOLUTION

Status: Resolved.

LOW **L-04** ACKNOWLEDGED

Lacking Reentrancy Checks

CATEGORY	Best Practices
LOCATION	`OrderHandler.sol`
REVIEW	Main Review

DESCRIPTION

In the new `executeOrderFromController` function there is no `nonReentrant` modifier to prevent re-entrancy back into the `executeOrderFromController` function.
For all functions on handler contracts which are not `globalNonReentrant`, they should be declared `nonReentrant`. Ideally all `globalNonReentrant` functions in a handler contract *are also* marked `nonReentrant`, so that there can be no re-entrancies that begin with a `globalNonReentrant` function and enter into a simple `nonReentrant` function.
While not strictly necessary, the appropriate re-entrancy guards for non `globalNonReentrant` functions will further constrict the allowed execution paths to only expected pathways.

RECOMMENDATION

Firstly, mark the `executeJitOrder` function as `nonReentrant`.
Secondly, consider making all handler functions `nonReentrant` in addition to `globalNonReentrant` so that the purely `nonReentrant` functions cannot be re-entered into from a `globalNonReentrant` function. This extra validation comes with a cost of an additional storage write which will negatively impact gas costs. To combat this, instead a transient storage re-entrancy guard can be used.

RESOLUTION

Status: Acknowledged.

LOW L-05

RESOLVED

Lacking From And To Market Check

CATEGORY	Validation
LOCATION	`GlvShiftUtils.sol`
REVIEW	Main Review

DESCRIPTION

The `validateGlvShift` function does not validate if the from market and to market are the same market. This could lead to unexpected issues if the `orderKeeper` accidentally initiates a shift where the from market is the same market as the to market.

RECOMMENDATION

Consider adding a validation to the `validateGlvShift` function that prevents the from and to markets from being the same address.

RESOLUTION

Status: Resolved.

LOW L-06

RESOLVED

Outdated Naming

CATEGORY	Documentation
LOCATION	`GlvShiftUtils.sol`
REVIEW	Main Review

DESCRIPTION

In the `validatePriceImpact` function, the function itself and several variables mention price impact. However the underlying key has been renamed to the `glvShiftMaxLossFactorKey`.

RECOMMENDATION

Consider renaming the `validatePriceImpact` function and the variables inside of it to be congruent with the new `glvShiftMaxLossFactorKey` name.

RESOLUTION

Status: Resolved.

LOW **L-07** ACKNOWLEDGED

Unexpected Behavior When GLV First Created

CATEGORY	Warning
LOCATION	Global
REVIEW	Main Review

DESCRIPTION

Because GLV now uses an oracle for prices, there will be some lag between the current GLV price and the price reported by the oracle. One particular case which causes a significant difference between these is when a GLV has been created and receives it's first deposit.
The initial deposit requires that a small amount of GLV is locked to set an initial non-trivial supply and backing. This is the first jump where GLV goes from being worth nothing to being worth something.
If the GLV oracle is already live at this point there may be unexpected behavior where in the block of the first deposit, and potentially in the subsequent blocks afterwards, the oracle continues to report GLV as being worth nothing while GLV is indeed worth something.
This may lead to unexpected edge cases which are not forseen, for example this causes a divide by zero revert in the `usdToGlvTokenAmount` function.

RECOMMENDATION

Be sure to only create the oracle for a GLV and allow it to be used after the GLV is established with some amount of non-trivial TVL for a sufficient duration.

RESOLUTION

Status: Acknowledged.

LOW L-08

RESOLVED

Max Pnl Validation Missed

CATEGORY	Validation
LOCATION	ExecuteGlvDepositUtils.sol
REVIEW	Main Review

DESCRIPTION

In the `_processMarketDeposit` function the `validateMaxPnl` validation only occurs if the deposit is a market token deposit. However this validation could be performed even if the GLV deposit is not a market token deposit, since it is bad for GLV holders to receive a GM token which is above it's max pnl factor for withdrawals regardless of how the depositor obtains them.
There is a clear incentive for a depositor to use the GLV as exit liquidity if they are already holding these GM tokens, however it's worthwhile to always prevent the illiquid tokens from winding up in GLV when the max pnl factor for withdrawals is exceeded.

RECOMMENDATION

Consider performing the `validateMaxPnl` always in the `_processMarketDeposit` function.

RESOLUTION

Status: Resolved.

LOW L-09

ACKNOWLEDGED

Increased Risk Of Arbitrage

CATEGORY	Gaming
LOCATION	Global
REVIEW	Main Review

DESCRIPTION

Since the GLV price is now determined by an oracle which carries a small delay, there is a risk of arbitraging the outdated oracle price relative to the real underlying value of GLV. This is because users can directly observe the real value of GLV in realtime onchain, and compare this to the reported price from the oracle. This same staleness applies to all other actions on GMX, and typically oracle delays are not a large concern because of fees applied to actions which use the oracle reference price. However on GLV deposits with GM tokens, there are no new fees which are levied on the user. This allows users to have much more leeway to create a profitable arbitrage if e.g. the oracle reports GM A as being worth \$1.50 while it is currently worth \$1.53.
Notice that this slightly better chance at realizing a profit from the slight oracle staleness only applies to users who are already holding GM tokens.

RECOMMENDATION

Consider requiring a lower tolerance for the maximum staleness of the GLV oracle if it is a dataStream, or use an aggregator with a very low deviation threshold.

RESOLUTION

Status: Acknowledged.

LOW **L-10** ACKNOWLEDGED

GlvShift Interval DoS

CATEGORY	DoS
LOCATION	`Global`
REVIEW	Main Review

DESCRIPTION

When a GLV Shift occurs a cooldown is reset which prevents further GLV shifts from occurring for a given period of time.
With the JIT feature, if this cooldown period is configured for a non-trivial amount of time, users could prevent legitimate rebalances or other JIT actions by consistently placing large trades and then closing them shortly thereafter.

RECOMMENDATION

Be aware of this DoS vector and ensure that the cooldown period for GLV shifts is assigned with this in mind.

RESOLUTION

Status: Acknowledged.

LOW **L-11** RESOLVED

Unclear getPrimaryPrice Behavior

CATEGORY	Documentation
LOCATION	`Oracle.sol`
REVIEW	Main Review

DESCRIPTION

The `getPrimaryPrice` function implementation documentation describes that it reverts when the price is empty for the given token.
However the `getPrimaryPrice` function does not revert when the token is the zero address and the price is instead returned as the empty price.

RECOMMENDATION

Either make this behavior abundantly clear with a warning or consider changing the behavior of the `_getPrimaryPrice` function to revert if `raiseOnEmpty` is true and the provided token is the zero address.

RESOLUTION

Status: Resolved.

LOW **L-12** ACKNOWLEDGED

GLV Value Updates Are Incorrect

CATEGORY	Logical Error
LOCATION	ExecuteGlvDepositUtils.sol
REVIEW	Main Review

DESCRIPTION

In the `executeGlvDeposit` function the `getGlvValue` function is used before and after the execution of the GLV deposit. Before the deposit the `getGlvValue` function is used to determine the GLV mint amount based on the current value of GLV and the value of the assets being deposited by the user.
After the deposit has been executed, the `getGlvValue` function is used a second time to query the updated value of GLV to emit with the `emitGlvValueUpdated` event.
However the `forceCalculation` boolean for this second invocation is false, and therefore it reports the same GLV price that was set by the oracle, showing no updated GLV value after the deposit.
Similarly during GLV withdrawals and shift flows the `getGlvValue` function is queried again to emit the updated value in the `emitGlvValueUpdated` event, though it inaccurately reports this because the `forceCalculation` is `false`.

RECOMMENDATION

Consider removing the `emitGlvValueUpdated` event from the GLV deposit, GLV withdrawal, and GLV shift flows.

RESOLUTION

Status: Acknowledged.

LOW **L-13** RESOLVED

Unexpected Key Replay

CATEGORY	Unexpected Behavior
LOCATION	`JitOrderHandler.sol`
REVIEW	Main Review

DESCRIPTION

In the `JitOrderHandler` `_createGlvShift` function the key of the GLV shift is based on the `orderKey` and the index of the shift action in the shift list. Keys are generally expected to be unique to each action and only correspond to one distinct action on the exchange.
However, it's possible for the same orders to be executed multiple times through the `JitOrderHandler` in the event that an order is initially frozen and then unfrozen by the user updating the order. As a result, the `_createGlvShift` can construct the same key for a corresponding GLV shift action, only this time the actual shift parameters of the from and to markets could be different.
This breaks the typical assumption that an action key is only executed once and that an action key pertains to one unique action on the exchange. In this case the ultimate effect is just that events will emit updates for this key multiple times across potentially different GLV shift actions.

RECOMMENDATION

Be aware of this behavior and if it is not desired, consider making the key unique to the GLV shift parameters so that the key must at least pertain to the same kind of shift. Or enforcing that it is unique to the JIT execution by hashing the `block.timestamp` with the key.

RESOLUTION

Status: Resolved.

LOW **L-14** RESOLVED

Shifts Occur No Matter The Order Result

CATEGORY	Unexpected Behavior
LOCATION	`JitOrderHandler.sol`
REVIEW	Main Review

DESCRIPTION

The `JitOrderHandler` relies on the `executeOrderFromController` for order execution. This `executeOrderFromController` function does not revert if the order fails to execute, and instead uses the `_handleOrderError` function of the OrderHandler to silently handle the cancellation or freezing of said order.
This results in shifts being executed and remaining executed in the event that an order executed through the `JitOrderHandler` fails execution. This may be unexpected since liquidity was moved without being utilized by the increase order.

RECOMMENDATION

Consider if this is the expected behavior. If not, consider reverting the `executeOrderFromController` in the event that the order execution fails.

RESOLUTION

Status: Resolved.

LOW L-15

RESOLVED

Multiple Shifts Cannot Occur

CATEGORY	Logical Error
LOCATION	`GlvShiftUtils.sol`
REVIEW	Main Review

DESCRIPTION

In the `executeGlvShift` function the `validateGlvShiftInterval` validation is performed to ensure that for a given GLV shifts are only executed at most once every `glvShiftMinInterval`.
However, in the context of a JIT order execution a list of shifts will be provided and all must be executed within the same GLV because the receiving GM market can only belong to one GLV token.
The list of shifts will not be able to be executed because the first shift action will set the `glvShiftLastExecutedAtKey` value to the current `block.timestamp`, causing the following shift executions to fail the `validateGlvShiftInterval` validation.

RECOMMENDATION

Consider either setting the `glvShiftMinInterval` to zero and retiring this validation, or allowing the validation to be bypassed by the JIT shift execution.

RESOLUTION

Status: Resolved.

LOW **L-16** RESOLVED

Lacking doExecuteGlvShift Reentrancy Guard

CATEGORY	Reentrancy
LOCATION	`GlvShiftHandler.sol`
REVIEW	Main Review

DESCRIPTION

In the `GlvShiftHandler` there is no reentrancy guard for the `doExecuteGlvShift` function which can by called by the contract itself or from a controller contract. A reentrancy guard could be added which would not clash with the `globalNonReentrant` of the `executeGlvShift` function and add some protection against any unexpected reentrancy into specifically the `doExecuteGlvShift` from outside controllers.

RECOMMENDATION

Consider adding a `nonReentrant` modifier to the `doExecuteGlvShift` function.

RESOLUTION

Status: Resolved.

LOW **L-17** ACKNOWLEDGED

Order Execution Feature For Liquidation Contract

CATEGORY	Unexpected Behavior
LOCATION	`LiquidationHandler.sol`
REVIEW	Main Review

DESCRIPTION

The `LiquidationHandler` contract validates the `executeOrderFeatureDisabledKey` however it uses the `address(this)` which means that the feature enabled status will be validated with the `LiquidationHandler` contract as the context. Therefore if the order feature is disabled for the `OrderHandler`, the order execution feature can still be enabled and orders can still be executed through the `LiquidationHandler`.

RECOMMENDATION

Be aware of this behavior.

RESOLUTION

Status: Acknowledged.

LOW L-18

ACKNOWLEDGED

Unnecessary GLV Price Timestamp Requirements

CATEGORY	Unexpected Behavior
LOCATION	`Global`
REVIEW	Main Review

DESCRIPTION

During the execution of a GLV deposit or a GLV withdrawal, the prices used by the oracle are required to be accurate for the market order timestamp rules. This includes the GLV price which will be submitted to the oracle but not used for the withdrawal or deposit sub action processing.
The GLV price is unnecessarily subject to the rules of the index, long, and short token prices that will be used for the deposit or withdrawal actions. This may cause issues in some cases where the most up to date price for GLV cannot be used to execute a GLV order that is somewhat stale, but still normally executable.
This gives more credence to the issue described in H-02, where in some cases keeper's may be required to submit a slightly outdated GLV price to meet these requirements.
The same applies to GLV shifts, where the order keeper may create a GLV shift and not immediately execute the request. In this case unnecessary constraints may apply to the GLV price submission upon execution of the GLV shift.

RECOMMENDATION

Keepers must be adept at understanding when to provide the GLV price from the data stream versus when to not provide the GLV price from the data stream when the latest most accurate price cannot satisfy the deposit and withdrawal market order price timestamp requirements.
Alternatively, similar to the solution of H-02, the computation could be forced for all GLV deposits and withdrawals and GLV shifts that are created in two-steps through the GLVShiftHandler.

RESOLUTION

Status: Acknowledged.

LOW L-19

ACKNOWLEDGED

Unexpected Order Cancellation

CATEGORY	Unexpected Behavior
LOCATION	`OrderUtils.sol`
REVIEW	Main Review

DESCRIPTION

In the `OrderUtils.cancelOrder` function the `isSupportedOrder` validation now optionally raises either the `UnsupportedOrderType` error or the `UnsupportedOrderTypeForAutoCancellation` error depending on if the cancellation being executed is an auto cancellation or not.
The `UnsupportedOrderType` error is specifically handled in the `OrderHandler._handleOrderError` function so that the execution reverts instead of going on to cancel the order. However the `UnsupportedOrderTypeForAutoCancellation` error is not explicitly handled this way.
This means that when an outdated `autoCancel` order using an old order type which is no longer supported is encountered, the order that triggers this autoCancellation will fail it's execution and be cancelled.
The alternative behavior, if there were similar specific handling for the `UnsupportedOrderTypeForAutoCancellation` error, would be that the order fails execution and simply reverts instead of being cancelled.

RECOMMENDATION

This behavior may be fine, though it differs from the handling of other `UnsupportedOrderType` failures. Be aware of this inconsistency and consider if this is the desired behavior when an outdated unsupported order type is encountered during auto cancellation.

RESOLUTION

Status: Acknowledged.

LOW L-20

ACKNOWLEDGED

JIT DoS'd With Atomic Actions

CATEGORY	DoS
LOCATION	Global
REVIEW	Main Review

DESCRIPTION

Though there is minimal incentive for a perpetrator, it is possible to DoS the execution of a JIT action or the possibility of such execution by making atomic actions to swap or withdraw the necessary backing liquidity that would allow the JIT to take place and occur within the acceptable impact threshold configured.

RECOMMENDATION

Be aware of this DoS vector, though it is unlikely to provide any benefit to the perpetrator at a non-trivial cost.

RESOLUTION

Status: Acknowledged.

LOW **L-21** RESOLVED

JIT Short Term Risk Free Trades

CATEGORY	Gaming
LOCATION	`JitOrderHandler.sol`
REVIEW	Main Review

DESCRIPTION

Due to the validations around the execution of JIT actions, it may be possible to carry out a short term trade depending on the keeper logic.
For example, it may be too price impactful to carry out a JIT action to execute a large market order order at t = 100. The user could observe that price moves in their favor by t = 110 and decide to make a swap in the GM from market to balance it, thus allowing the JIT action to be within the overall price impact constraints. Then and only then is the JIT action carried out, using the prices adhering to the market order requirements at t = 100.
This attack class applies to any validations that would prevent the JIT action from taking place, such as lacking GLV liquidity to shift, too significant resulting OI imbalance, too high specific GM concentration in a GLV, etc…

RECOMMENDATION

Rather than waiting to see if conditions improve to allow a JIT action to occur, execute an order normally to cancel or freeze the order immediately.

RESOLUTION

Status: Resolved.

LOW L-22

ACKNOWLEDGED

JIT May Cause Some Orders To Fail

CATEGORY	Unexpected Behavior
LOCATION	`JitOrderHandler.sol`
REVIEW	Main Review

DESCRIPTION

If the increase order has a `swapPath` which goes through a GM market that the JIT action pulls liquidity from, the JIT action itself could cause the order execution to fail.
This may be unexpected and in some cases may unnecessarily cause orders to be cancelled rather than executed.

RECOMMENDATION

If there is ever an option between using a GM market which is not in the order's `swapPath` and using one which is in the order's `swapPath` for a JIT action, the GM market which is not in the order's `swapPath` should be used.

RESOLUTION

Status: Acknowledged.

LOW L-23

ACKNOWLEDGED

JIT May Prevent Withdrawals

CATEGORY	Warning
LOCATION	`Global`
REVIEW	Main Review

DESCRIPTION

Because GLV withdrawals are two step actions there may be a certain amount of GM assets that have open requests to be withdrawn from a GLV by GLV holders.
If a GLVShift as a result of a JIT action occurs before these GLV withdrawals are executed then it may cause these withdrawals to fail as it removes the specific GM tokens that these withdrawals requested.

RECOMMENDATION

Be aware of this risk of DoSing in-flight withdrawals and consider having the keeper take into account open withdrawal requests before initiating a JIT action.

RESOLUTION

Status: Acknowledged.

LOW L-24

ACKNOWLEDGED

GLV May Be Left Improperly Allocated

CATEGORY	Warning
LOCATION	`Global`
REVIEW	Main Review

DESCRIPTION

If after a JIT action is executed a user closes their position shortly after it is successfully opened, the GLV may be left improperly allocated in a market that doesn't hold much open interest.
This will require another GLV Shift to shift back to a market with high utilization, which will ultimately cost more slippage upon the execution of this additional shift.
Overtime this increased shifting may have a negative impact on GLV's value accrual.

RECOMMENDATION

Be aware of this increased cost of additional shifts when markets quickly lose open interest due to concentrated positions as a result of JIT.

RESOLUTION

Status: Acknowledged.

LOW **L-25** RESOLVED

Unnecessary ReentrancyGuard

CATEGORY	Superfluous Code
LOCATION	`JitOrderHandler.sol`
REVIEW	Main Review

DESCRIPTION

The `JitOrderHandler` contract inherits from the OpenZeppelin `ReentrancyGuard` abstract contract, however does not use any of the `ReentrancyGuard` features.
Instead the `JitOrderHandler` makes use of the `globalNonReentrant` modifier which does not come from the `ReentrancyGuard` contract.

RECOMMENDATION

Remove the `ReentrancyGuard` contract import and usage in the `JitOrderHandler` contract.

RESOLUTION

Status: Resolved.

LOW **L-26** ACKNOWLEDGED

Multiple Shifts From A Market

CATEGORY	Validation
LOCATION	`JitOrderHandler.sol`
REVIEW	Main Review

DESCRIPTION

In the `_processShifts` function there is no validation that ensures that a shift from a `fromMarket` has not already occurred.
It doesn't make sense to shift from the same `fromMarket` twice since this could occur in one larger shift, therefore this unexpected execution path could be limited to avoid unexpected behavior or logical issues from the keeper.

RECOMMENDATION

Consider adding validation that checks if a market exists in the `shiftParamsList` as a `fromMarket` more than one time and reverts if such a market is found.

RESOLUTION

Status: Acknowledged.

LOW L-27

RESOLVED

Unnecessary Library Usage

CATEGORY	Gas Optimization
LOCATION	`JitOrderHandler.sol`
REVIEW	Main Review

DESCRIPTION

In the `JitOrderHandler` contract the following library usages are unnecessary:
- `using GlvDeposit for GlvDeposit.Props`
- `using GlvShift for GlvShift.Props`
- `using GlvWithdrawal for GlvWithdrawal.Props`

RECOMMENDATION

Remove these unnecessary library usages.

RESOLUTION

Status: Resolved.

LOW **L-28** RESOLVED

Simulation Misses Reentrancy Guard Close

CATEGORY	Unexpected Behavior
LOCATION	`JitOrderHandler.sol`
REVIEW	Main Review

DESCRIPTION

In the `simulateExecuteJitOrder` function the `globalNonReentrant` modifier comes before the `withSimulatedOraclePrices` modifier.
As a result the `_globalNonReentrantAfter` block is unreachable because the `withSimulatedOraclePrices` ending reverts before this is reached.
The net impact is just that the simulation may show an inaccurate amount of gas used, since there would be some gas refund for resetting the `REENTRANCY_GUARD_STATUS` to `NOT_ENTERED`.
Furthermore, this contradicts the ordering that has been used throughout the GMX codebase for other simulation functions.

RECOMMENDATION

Consider updating the ordering of the two modifiers such that the `withSimulatedOraclePrices` modifier comes before the `globalNonReentrant` modifier.

RESOLUTION

Status: Resolved.

MEDIUM **M-01** RESOLVED

GLV Shift Misses Feature Validation

CATEGORY	Validation
LOCATION	`GlvShiftUtils.sol`
REVIEW	Remediation Review 1

DESCRIPTION

The `executeGlvShift` function directly uses the `ShiftUtils.executeShift` function to execute the underlying shift action, however does not validate if the `executeShiftFeatureDisabledKey` feature is disabled for the `ShiftHandler` address.
Therefore if shifts were to be disabled they could still be executed through the `GLVShift` action.

RECOMMENDATION

Consider implementing and using an `executeShiftFromController` function which similar to other functions, validates the feature and performs the shift action.

RESOLUTION

Status: Resolved.

LOW **L-01** RESOLVED

Remaining ReentrancyGuard Import

CATEGORY	Best Practices
LOCATION	`JitOrderHandler.sol`
REVIEW	Remediation Review 1

DESCRIPTION

In the `JitOrderHandler` contract there is a `ReentrancyGuard` import that is unused.

RECOMMENDATION

Consider removing this unused import.

RESOLUTION

Status: Resolved.

LOW **L-02**

PARTIALLY RESOLVED

GLV Oracle Inaccuracy

CATEGORY	Logical Error
LOCATION	`Global`
REVIEW	Remediation Review 1

DESCRIPTION

Throughout the different actions that can use the new GLV price oracle, none of them account for the GLV price oracle being used in the relevant `estimateOraclePriceCount` functions.
This means that actions submitted for `glvDeposit`, `glvWithdrawal`, and orders that would use JIT liquidity do not accurately account for the GLV price oracle being used.
In actions like glvDeposit and glvWithdrawal the GLV price oracle may replace several price oracles and therefore the oracle price count is over estimated.
For JIT orders the oracle price count would be under estimated since the GLV oracle price would be additional to the prices required by the normal order execution flow and oracle count estimation for the order.

RECOMMENDATION

Be aware of this inconsistency in the estimated oracle price count in GLV actions and JIT actions.

RESOLUTION

Status: Partially resolved.

LOW L-03

ACKNOWLEDGED

onlySelfOrController Unnecessary

CATEGORY	Superfluous Code
LOCATION	Global
REVIEW	Remediation Review 1

DESCRIPTION

Technically, the `onlySelfOrController` modifier can simply be replaced by the `onlyController` modifier, since it is always used by contracts which will hold the controller role.

RECOMMENDATION

Consider removing the unnecessary `onlySelfOrController` modifier in favor of the already used `onlyController` modifier.

RESOLUTION

Status: Acknowledged.

LOW L-04

ACKNOWLEDGED

Integration Risk During Upgrade

CATEGORY	Warning
LOCATION	`Global`
REVIEW	Remediation Review 1

DESCRIPTION

During the upgrade period for the new GMX Shift and JIT updates, there may be multiple handlers active at the same time which means deposits, shifts, glv deposits, glv withdrawals, and orders could all have executions and callbacks coming from multiple handler contracts.
For some integrations this may pose an issue.

RECOMMENDATION

Be sure to alert integrations about this upgrade in advanced.

RESOLUTION

Status: Acknowledged.

LOW L-05

ACKNOWLEDGED

Increased Hedge Risk For GLV

CATEGORY	Warning
LOCATION	Global
REVIEW	Remediation Review 1

DESCRIPTION

With the introduction of JIT, there is an increased risk that GLV is exposed to unhedged market exposure, since often the act of opening a large trade that requires new liquidity to back it will create an imbalance in the market that may or may not be filled.

RECOMMENDATION

Be aware of this increased risk of unhedged exposure that may be brought to GLV, and consider adding validations in the keeper to ensure that the resulting state of the market will not be skewed above an acceptable threshold either net long or short.

RESOLUTION

Status: Acknowledged.

MEDIUM **M-01** ACKNOWLEDGED

Manual Cancellation Sends Refund To Unexpected Address

CATEGORY	Unexpected Behavior
LOCATION	`Global`
REVIEW	Remediation Review 3

DESCRIPTION

In the manual cancellation flow the account of the relevant action is set as the keeper, therefore sending what is estimates as the keeper fee to the account as a refund. This includes the estimated costs for oracle prices, base execution fee amounts, and gas multipliers.
However, the action may have had an associated callback contract or dedicated cancellation receiver/refund receiver which now does not receive these funds which should have been technically refunded to that separate account.

RECOMMENDATION

Be aware of this behavior and document it for integrators, otherwise consider putting whichever address that would be provided as a `refundReceiver` as the keeper for the manual cancellation flow.

RESOLUTION

Status: Acknowledged.

LOW **L-01** ACKNOWLEDGED

Swap Oracle Requirement Understated

CATEGORY	Warning
LOCATION	Global
REVIEW	Remediation Review 3

DESCRIPTION

In the `estimateOraclePriceCount` functions the `swapCount` only introduces 1 oracle count per swap, however a swap will usually necessitate 2 new prices: 1 for the index token of the market and 1 for the opposite token which has not already been recorded in the base market or a previous swap path market.

RECOMMENDATION

Be aware of this underallocation and consider if it should be adjusted to 2 prices per swap count.

RESOLUTION

Status: Acknowledged.

INFORMATIONAL I-01 ACKNOWLEDGED

Single Market GLVs Do Not Collect Enough Fees

CATEGORY	Warning
LOCATION	Global
REVIEW	Remediation Review 3

DESCRIPTION

In the `estimateGlvDepositOraclePriceCount` and `estimateGlvWithdrawalOraclePriceCount` functions the base oracle count when
`glvTokenPriceUsed` is true is 4.
However for GLVs with only one market, there can be only 3 required oracle prices for that GLV action. Therefore specifying `glvTokenPriceUsed` as false upon creation allows the user to potentially provide less executionFee than should be necessary if the
`glvTokenPriceUsed` of `true` is used during execution.

RECOMMENDATION

This may be safely acknowledged as GLVs with one market should not exist, however be aware of this quirk.

RESOLUTION

Status: Acknowledged.

§ 05

Disclaimer.

This report is not, nor should be considered, an "endorsement" or "disapproval" of any particular project or team. This report is not, nor should be considered, an indication of the economics or value of any "product" or "asset" created by any team or project that contracts Guardian to perform a security assessment. This report does not provide any warranty or guarantee regarding the absolute bug-free nature of the technology analyzed, nor do they provide any indication of the technologies proprietors, business, business model or legal compliance.

This report should not be used in any way to make decisions around investment or involvement with any particular project. This report in no way provides investment advice, nor should be leveraged as investment advice of any sort. This report represents an extensive assessing process intending to help our customers increase the quality of their code while reducing the high level of risk presented by cryptographic tokens and blockchain technology.

Blockchain technology and cryptographic assets present a high level of ongoing risk. Guardian's position is that each company and individual are responsible for their own due diligence and continuous security. Guardian's goal is to help reduce the attack vectors and the high level of variance associated with utilizing new and consistently changing technologies, and in no way claims any guarantee of security or functionality of the technology we agree to analyze.

The assessment services provided by Guardian is subject to dependencies and under continuing development. You agree that your access and/or use, including but not limited to any services, reports, and materials, will be at your sole risk on an as-is, where-is, and as-available basis.

Cryptographic tokens are emergent technologies and carry with them high levels of technical risk and uncertainty. The assessment reports could include false positives, false negatives, and other unpredictable results. The services may access, and depend upon, multiple layers of third-parties.

Notice that smart contracts deployed on the blockchain are not resistant from internal/external exploit. Notice that active smart contract owner privileges constitute an elevated impact to any smart contract's safety and security. Therefore, Guardian does not guarantee the explicit security of the audited smart contract, regardless of the verdict.

§ 06

About Guardian.

Guardian is a smart contract security firm. Every engagement is worked by two independent teams competing on the same scope, each with its own tooling and methods, so no single reviewer's blind spot survives.

$45B+
DIGITAL ASSETS UNDER REVIEW

200+
ENGAGEMENTS COMPLETED

500+
CRITICAL & HIGH FINDINGS REPORTED

Reach us at audits@guardianaudits.com.